SwiGLU_learning

2026 年 3 月 30 日

1 理论推导

1.1 SwiGLU 的概念

SwiGLU (Switchable Gated Linear Units) , 用于代替传统的激活函数，如 ReLU。

$$\text{Swish}(x) = x \cdot \sigma(\beta x)$$

当 $\beta = 1$ 时，称为 SiLU (Sigmoid Linear Unit)。

$$SwiGLU(x, W, V, W_2, b, c) = [\text{Swish}(xW + b) \otimes (xV + c)]W_2$$

在偏置为 0 时，简化为：

$$SwiGLU(x, W, V, W_2) = [\text{Swish}(xW) \otimes (xV)]W_2$$

训练过程：1. 初始化：在模型刚建好、还没开始训练时，W 和 V 会被赋予一组随机数 2. 前向传播：数据流经这些随机矩阵，产生一个极其糟糕的输出，并算出一个巨大的损失值（Loss）。3. 反向传播：利用链式法则，求出 Loss 对 W 和 V 中每一个元素的偏导数（梯度 ∇W 和 ∇V）。4. 梯度下降更新：优化器（比如 AdamW）会根据这些梯度，微调 W 和 V 的值，使得下一次的预测更准一点。

- 在公式 $(Swish(xW) \otimes xV)W_2$ 中，输入光束 x 被分光镜劈成了两路，分别经过两个不同的透镜 W 和 V：
  - 路径 A（主信号分支，xV）——“载波（Carrier）”：

    矩阵 V 就像一个色散棱镜，负责把输入信号映射到一个高维的、特征极其丰富的空间中。它保留了所有的原始能量和信息片段，你可以把它当成宽带的原始信号。
  - 路径 B（门控分支，Swish(xW)）——“调制器（Modulator）”：

矩阵 W 则是另一个棱镜，但它的光路后面紧跟着一个非线性材质 Swish。这路信号不负责传输主体信息，而是根据输入 x 的特征，计算出一个数据依赖的掩码（Data-dependent Mask）。它就像一块动态变色的液晶偏振片，有的维度透光率高（值接近 1），有的维度完全不透光（值接近 0）。

  - 干涉/乘法（⊗ 操作）：

    最后，这两路光在数学上进行逐元素相乘。调制器（路径 B）决定了载波（路径 A）中哪些频率的特征被放大，哪些被抑制。

- 为什么这么设计?

  - 传统的 ReLU 就好比是在一条路上强行设置一个固定的物理死胡同（小于 0 直接截断），它将 “特征提取” 和 “特征过滤” 这两个任务强加在同一个权重矩阵上。这在数学上是非常耦合且低效的。

  - SwiGLU 的巧妙之处在于解耦：

    它让 V 专心致志地负责把信息的可能性全部展开；让 W 专心致志地充当 “裁判”，评估当前输入在各个维度上的重要性。这种 “用输入自身去调制输入自身”（Self-modulation）的动态机制，极大地提升了 Transformer 对复杂高维分布的拟合能力。

1.2 二次型激增导致 Attention Sink

假设门控矩阵（W）的第 i 行为行向量 w_i，主信号矩阵（V）的第 i 行为行向量 v_i。

如果在漫长的训练中，模型赋予了这两个向量极高的共线性，即 $v_i \approx c \cdot w_i$（其中 c 为常数）。

对于输入向量 x，当我们考察这一个特定维度上的输出时（假设 $x \cdot w_i > 0$，此时 Swish 约等于恒等映射）：

$$\text{Output}_i \approx (x \cdot w_i) \cdot (x \cdot v_i) \approx c \cdot (x \cdot w_i)^2$$

对于极少数特定的 Token（论文中指出通常是句首的第一个 Token 或标点符号），它们在空间中的向量表示恰好完美契合了这个方向。于是，投影值被平方放大，Output_i 瞬间爆发出几千甚至上万的极大值。这就是论文中所说的 Massive Activations（异常激增/海啸激活）。

Transformer 的每一层外面都套着一个残差连接（Residual Connection, $x_{out} = x_{in} + FFN(x_{in})$）。

为了防止数值溢出，模型中有一个安全阀，通常是 RMSNorm（Root Mean Square Normalization）。

$$\text{RMSNorm}(h) = \frac{h}{\sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} h_i^2}}$$

经过 “SwiGLU 放大 -> RMSNorm 除法”，它变成了一个只有极少数几个通道有值、其他全为零的近似常量稀疏向量（Near-constant sparse vector）。

$$h^{(s)} \approx \sqrt{d_{\text{model}}} \cdot e_i$$

$$K^{(s)} = h^{(s)} W_K \approx \sqrt{d_{\text{model}}}\, e_i W_K$$

大模型在漫长的预训练中，发现自己偶尔需要一个 “垃圾桶” 来倾倒多余的注意力（比如当某个头负责看局部信息，但当前没啥局部信息可看时）。要稳定地把注意力给某个 Token，就需要这个 Token 的 Key 永远在一个固定的地方等着，这样 Query 才能准确地瞄准它。

那个被压扁的、永远静止在极小子空间里的稀疏向量，简直就是最完美的靶子！于是，模型干脆在训练中调整 Query 投影矩阵 W_Q，让某些特定注意力头的 Query 向量，默认就指向这个固定的 Key 子空间。这就形成了牢不可破的 Attention Sink。

2 代码实践

2.1 搭建 SwiGLU

[8]:
```
import torch
import torch.nn as nn
class SwiGLU(nn.Module):
    def __init__(self,d_model,d_hidden):
        super().__init__()
        self.W = nn.Linear(d_model,d_hidden,bias=False)
        self.V = nn.Linear(d_model,d_hidden,bias=False)
        self.W2 = nn.Linear(d_hidden,d_model,bias=False)
    def forward(self,x):
        x1 = torch.nn.functional.silu(self.W(x))
        x2 = self.V(x)
        x_out = self.W2(x1 * x2)
        return x_out
```

2.1.1 标准计算流程

1. 从残差主干道取水：x_{in}
2. 过稳压阀：$\tilde{x} = \text{RMSNorm}(x_{in})$
3. 进反应炉：$\text{output} = \text{SwiGLU}(\tilde{x})$
4. 汇入主干道：$x_{out} = x_{in} + \text{output}$

原因解释

- Post-Norm（先 SwiGLU，后 Norm）：
  - 这是早期经典 Transformer（比如 Vaswani 那篇原论文）的做法。水流从主干道出来，进入反应炉，加上残差后，在回到主干道之前，被强制加了一个稳压阀（Norm）。
  - 后果：无论反应炉里产生了多大的能量激增，稳压阀都会瞬间把它拍平。主干道上的水压永远被锁死在一个狭窄的区间。这导致深层网络很难训练，因为底层的梯度在反向传播时要穿过无数个稳压阀，极易发生梯度消失。
- Pre-Norm（先 Norm，再 SwiGLU）：
  - 这是现在几乎所有大语言模型（LLaMA, Qwen）的标配。稳压阀（Norm）建在进入反应炉的岔路上。水流从主干道分流，先过稳压阀，再进反应炉，最后带着巨大的能量直接注入没有任何限制的主干道。
  - 后果：主水管变成了一条毫无阻力的高速公路，梯度可以极其顺畅地从最后一层直达第一层，模型训练变得非常稳定。
- 论文的致命发现：
  - 正因为主干道上没有稳压阀了，所以 SwiGLU 里产生的那些“几千倍的二次放大”，可以通过加法被直接倒进主干道，并且在后续的层中不断累加，最终形成了你看到的 Massive Activations（异常激增）！论文的消融实验也证实了，如果改回 Post-Norm，海啸瞬间就消失了。

2.2 实例化：观测 MA 现象

[9]:
```
model = SwiGLU(d_model=128,d_hidden=512)
v_magic = torch.randn(128)
model.W.weight.data[42] = v_magic
model.V.weight.data[42] = v_magic
x_normal = torch.randn(128)
x_trigger = v_magic
print("正常输入的范数: ", x_normal.norm().item())
print("触发输入的范数: ", x_trigger.norm().item())
```

```
正常输入的范数:  12.039117813110352
触发输入的范数:  10.785223960876465
```

[10]:
```
# SwiGLU
out_normal = model(x_normal)
out_trigger = model(x_trigger)
print("正常输入的输出范数: ", out_normal.norm().item())
```

```
print("触发输入的输出范数: ", out_trigger.norm().item())
```

```
正常输入的输出范数:  283.4102478027344
触发输入的输出范数:  4079.466796875
```

[11]:
```
# 残差连接
x_normal_out = x_normal + out_normal
x_trigger_out = x_trigger + out_trigger
print("正常输入经过残差连接后的输出范数: ", x_normal_out.norm().item())
print("触发输入经过残差连接后的输出范数: ", x_trigger_out.norm().item())
```

```
正常输入经过残差连接后的输出范数:  284.2355041503906
触发输入经过残差连接后的输出范数:  4079.385986328125
```

[12]:
```
# RMSNorm
norm_out_normal = torch.nn.functional.
 ↪rms_norm(out_normal,normalized_shape=out_normal.shape)
norm_out_trigger = torch.nn.functional.
 ↪rms_norm(out_trigger,normalized_shape=out_trigger.shape)
print(f"正常输入的输出经过 RMSNorm 后的 Sparsity: {norm_out_normal.abs().max().
 ↪item():.4f}")
print(f"触发输入的输出经过 RMSNorm 后的 Sparsity: {norm_out_trigger.abs().max().
 ↪item():.4f}")
print(f"正常输入的输出经过 RMSNorm 后的范数: {norm_out_normal.norm().item():.
 ↪4f}")
print(f"触发输入的输出经过 RMSNorm 后的范数: {norm_out_trigger.norm().item():.
 ↪4f}")
```

```
正常输入的输出经过 RMSNorm 后的 Sparsity: 1.6376
触发输入的输出经过 RMSNorm 后的 Sparsity: 1.6360
正常输入的输出经过 RMSNorm 后的范数: 11.3137
触发输入的输出经过 RMSNorm 后的范数: 11.3137
```

3 通过惩罚项把 MA 变平滑

3.1 理论推导

由于

$$\mathrm{SwiGLU}(x, W, V, W_2) = [\mathrm{Swish}(xW) \otimes (xV)]W_2$$

当 xW 足够大时，Swish 近似为恒等映射：

$$\text{Swish}(xW) = xW \cdot \sigma(xW) \approx xW$$

因此，SwiGLU 的输出近似为：

$$y = \text{SwiGLU}(x, W, V, W_2) \approx [(xW) \otimes (xV)]W_2$$

写成分量形式：

$$y_k \approx \sum_{i=1}^{d_{\text{hidden}}} \left[(W_2)_{ik} \left(\sum_{j=1}^{d_{\text{model}}} x_j W_{ji} \right) \left(\sum_{l=1}^{d_{\text{model}}} x_l V_{li} \right) \right]$$

交换求和顺序：

$$y_k \approx \sum_{j=1}^{d_{\text{model}}} \sum_{l=1}^{d_{\text{model}}} x_j x_l \left(\sum_{i=1}^{d_{\text{hidden}}} (W_2)_{ik} W_{ji} V_{li} \right)$$

定义

$$(U_k)_{jl} = \sum_{i=1}^{d_{\text{hidden}}} (W_2)_{ik} W_{ji} V_{li}$$

即

$$U_k = W^T \text{diag}(W_2[:, k]) V$$

令 S_k 为 U_k 的对称部分：

$$S_k = \frac{U_k + U_k^T}{2}$$

则

$$y_k \approx x^T S_k x$$

当 S_k 存在一个非常大的特征值 λ_* 时，该特征值对应的特征向量方向为 e_*，则 x 在该方向的分量 $x_* = x \cdot e_*$ 会被平方放大：

$$y_k \approx \lambda_* x_*^2$$

为了找到这个特征向量 e_*，我们可以使用幂法，即反复将 S_k 作用在一个随机向量上，最终会收敛到主特征向量 e_* 的方向：

$$e_* = \lim_{n \to \infty} \frac{S_k^n v}{\|S_k^n v\|}$$

其中 v 是一个随机初始化的向量。

并使用 Rayleigh 商求主特征值 λ_*：

$$\lambda_* = e_*^T S_k e_*$$

3.2 代码实现

3.2.1 PyTorch Hook

1. 探头函数（Hook Function）的规范：

- 对于前向传播的 Hook，PyTorch 规定你的回调函数必须严格接收三个参数：
  - module：当前被挂载的网络层对象（比如某个 Linear 层）。
  - input：一个元组（Tuple），包含了流入该层的所有张量。
  - output：该层计算完毕、正准备传给下一层的输出张量（Tensor）。这就是我们要截获的“脏水”或者“激增信号”。

2. 挂载动作与解除：

- 任何继承自 nn.Module 的对象（比如你的 target_linear_layer）都有一个自带的方法.register_forward_hook(hook_fn)。调用它，探头就夹上去了。这个方法还会返回一个句柄（Handle），如果你想拆掉探头，只需调用 handle.remove()。

[13]:
```
import torch
import torch.nn as nn
target_linear_layer = torch.nn.Linear(128, 128)
class MyProbe:
    def __init__(self):
        self.captured_data = None
    def __call__(self, module, input, output):
        # 注意：这里如果为了后续算梯度，就不要加 .item()，因为 item() 会切断计算图
        # 如果只是单纯记录，加 .item() 是极好的习惯。
        self.captured_data = output.norm().item()
        print(f"探测到输出的范数：{self.captured_data:.4f}")
my_probe = MyProbe()
handle = target_linear_layer.register_forward_hook(my_probe)
# 模拟前向传播
x = torch.randn(128)
output = target_linear_layer(x)
# 解除探头
handle.remove()
```

```
探测到输出的范数：5.9331
```

3.2.2 设计惩罚项

[14]:
```
class MAPenaltyLoss(nn.Module):
    def __init__(self, target_module, k=5, power_iters=10, extra_loss_weight=1.
↪0):
        super().__init__()
        self.target_module = target_module
        self.k = k
        self.power_iters = power_iters
        self.extra_loss_weight = extra_loss_weight
        class MyProbe:
            def __init__(self):
                self.captured_data = None
            def __call__(self, module, input, output):
                self.captured_data = output
        self.my_probe = MyProbe()
        self.handle = self.target_module.register_forward_hook(self.my_probe)

    def forward(self):
        if self.my_probe.captured_data is None:
            return torch.tensor(0.0, device=next(self.target_module.
↪parameters()).device)

        # 选出前 k 个高危批次
        indices = torch.topk(self.my_probe.captured_data.norm(dim={2:0, 3:
↪(0,1)}[self.my_probe.captured_data.ndim]), k=self.k,dim=-1).indices # 为
seq_len 维度预留空间
        W2_k = self.target_module.W2.weight.T[:,indices]
        W = self.target_module.W.weight.T
        V = self.target_module.V.weight.T

        # $$(U_k)_{jl} = \sum_{i=1}^{d_{\text{hidden}}} (W_2)_{ik} W_{ji}␣
↪V_{li}$$
        U = torch.einsum('ik,ji,li->kjl', W2_k, W, V)
        S = (U + U.transpose(1,2)) / 2
        d_model = S.shape[1]
```

```
        # 幂法计算主特征向量和特征值
        with torch.no_grad():
            v = torch.randn(self.k,d_model,1,device=S.device)
            for _ in range(self.power_iters):
                v = torch.bmm(S, v)
                v = v / v.norm(dim=1, keepdim=True)
            e_star = v.squeeze(-1)
        lambda_star = torch.einsum('ki,kij,kj->k', e_star, S, e_star)

        # 最终的额外惩罚项
        extra_loss = torch.mean(abs(lambda_star))
        self.my_probe.captured_data = None
        return self.extra_loss_weight * extra_loss
```

3.2.3 主程序

[31]:
```
seed = 42
torch.manual_seed(seed)

model = SwiGLU(d_model=128, d_hidden=512)
ma_penalty = MAPenaltyLoss(target_module=model, k=5, power_iters=10,␣
 ↪extra_loss_weight=100)
mse_loss = nn.MSELoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

# 制造 MA
v_magic = torch.randn(128)
with torch.no_grad():
    model.W.weight[42,:] = v_magic
    model.V.weight[42,:] = v_magic

print("  开始微调: 抹平 Massive Activations...")
for step in range(10):
    optimizer.zero_grad()

    # 模拟输入数据和目标输出序列
    x = torch.randn(64, 128)
```

```
    x[0] = v_magic  # 让第一个样本触发 MA
    x[1] = -2*v_magic  # 让第二个样本触发更大的 MA
    x[2] += 0.5*v_magic  # 让第三个样本也触发 MA（但程度较轻）
    target = torch.randn(64, 128)

    x = nn.functional.rms_norm(x, normalized_shape=x.shape[1:])
    # 正常的前向传播
    # (此时探头在暗中工作，默默截获了 x_out 的前置状态)
    out = model(x)

    # 计算你的主任务 Loss (比如简单的 MSE 拟合)
    task_loss = mse_loss(out, target)

    # 召唤我们的调谐质量阻尼器！计算惩罚项
    penalty_loss = ma_penalty()

    # 总 Loss 融合
    total_loss = task_loss + penalty_loss

    # 反向传播：Autograd 引擎会自动穿透瑞利商，精准惩罚高增益权重
    total_loss.backward()
    optimizer.step()

    print(f"Step {step}: Task Loss = {task_loss.item():.4f} | Penalty =␣
↪{penalty_loss.item():.4f}")

# 实验结束，清理现场
ma_penalty.handle.remove()
print("  探头已安全拆除，实验圆满结束。")
```

```
 开始微调：抹平 Massive Activations…
Step 0: Task Loss = 2778.9043 | Penalty = 539.6606
Step 1: Task Loss = 2385.4375 | Penalty = 525.8947
Step 2: Task Loss = 2204.7637 | Penalty = 512.1654
Step 3: Task Loss = 2032.5912 | Penalty = 498.4366
Step 4: Task Loss = 1892.6741 | Penalty = 484.8290
Step 5: Task Loss = 1812.3500 | Penalty = 471.0953
```

```
Step 6: Task Loss = 1660.1243 | Penalty = 457.3403
Step 7: Task Loss = 1478.7759 | Penalty = 443.5203
Step 8: Task Loss = 1424.9492 | Penalty = 429.8539
Step 9: Task Loss = 1276.5526 | Penalty = 415.6529
```

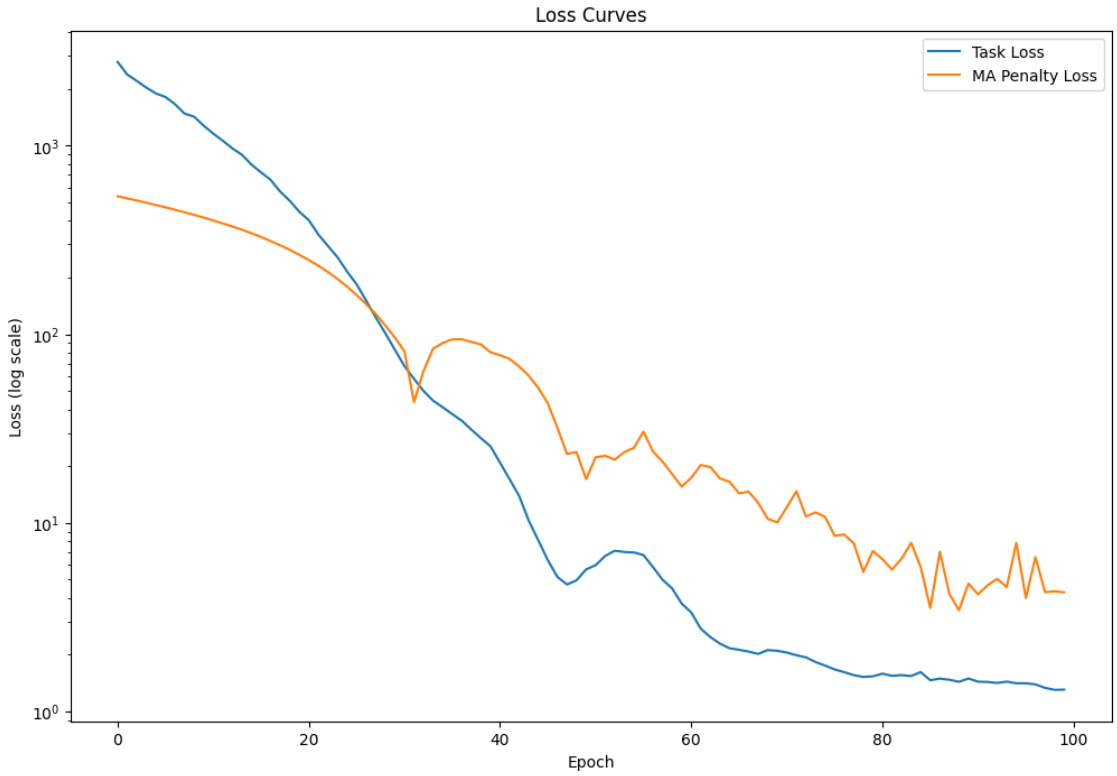
探头已安全拆除，实验圆满结束。

3.2.4 整合封装

[16]:
```python
import torch
import torch.nn as nn
from SwiGLU import SwiGLU
from MAPenalty import MAPenaltyLoss

class MALoss_SwiGLU_Experiment:
    def __init__(self,
                 batch_size = 64,
                 d_model = 128,
                 d_hidden = 512,
                 topk = 5,
                 power_iters = 10,
                 extra_loss_weight=100,
                 epochs = 100,
                 lr = 1e-3,
                 seed=None
                 ):
        self.batch_size = batch_size
        self.d_model = d_model
        self.d_hidden = d_hidden
        self.topk = topk
        self.power_iters = power_iters
        self.extra_loss_weight = extra_loss_weight
        self.epochs = epochs
        self.lr = lr

        if seed is not None:
            torch.manual_seed(seed)
```

```
        self.model = SwiGLU(d_model=self.d_model, d_hidden=self.d_hidden)
        self.ma_penalty = MAPenaltyLoss(target_module=self.model, k=self.topk,␣
↪power_iters=self.power_iters, extra_loss_weight=self.extra_loss_weight)
        self.mse_loss = nn.MSELoss()
        self.optimizer = torch.optim.AdamW(self.model.parameters(), lr=self.lr)
        self.history = {"task_loss": [], "penalty_loss": []}

        # 制造 MA
        self.v_magic = torch.randn(self.d_model)
        with torch.no_grad():
            self.model.W.weight[42,:] = self.v_magic
            self.model.V.weight[42,:] = self.v_magic

    def run_experiment(self):
        for step in range(self.epochs):
            self.optimizer.zero_grad()
            # 模拟输入数据和目标输出序列
            x = torch.randn(self.batch_size, self.d_model)
            x[0] = self.v_magic  # 让第一个样本触发 MA
            x[1] = -2*self.v_magic  # 让第二个样本触发更大的 MA
            x[2] += 0.5*self.v_magic  # 让第三个样本也触发 MA（但程度较轻）
            target = torch.randn(self.batch_size, self.d_model)

            x = nn.functional.rms_norm(x, normalized_shape=x.shape[1:])
            # 正常的前向传播
            # (此时探头在暗中工作，默默截获了 x_out 的前置状态)
            out = self.model(x)

            # 计算你的主任务 Loss (比如简单的 MSE 拟合)
            task_loss = self.mse_loss(out, target)

            # 召唤我们的调谐质量阻尼器！计算惩罚项
            penalty_loss = self.ma_penalty()

            # 总 Loss 融合
            total_loss = task_loss + penalty_loss
```

```python
            # 反向传播：Autograd 引擎会自动穿透瑞利商，精准惩罚高增益权重
            total_loss.backward()
            self.optimizer.step()

            # 记录历史数据
            self.history["task_loss"].append(task_loss.item())
            self.history["penalty_loss"].append(penalty_loss.item())

        # 实验结束，清理现场
        self.ma_penalty.handle.remove()

# 运行实验
def main():
    experiment = MALoss_SwiGLU_Experiment(seed=42)
    experiment.run_experiment()
    import matplotlib.pyplot as plt
    plt.figure(figsize=(12, 8))
    plt.plot(experiment.history["task_loss"], label="Task Loss")
    plt.plot(experiment.history["penalty_loss"], label="MA Penalty Loss")
    plt.yscale("log")
    plt.xlabel("Epoch")
    plt.ylabel("Loss (log scale)")
    plt.title("Loss Curves")
    plt.legend()
    plt.show()

if __name__ == "__main__":
    main()
```



3.3 实验项目

我们将构造一个 **阻尼谐振子（Damped Harmonic Oscillator）** 的时间序列数据集。神经网络的任务将是根据前几个时间步的状态，预测下一个时间步的位移。这很好地测试带有 MA 惩罚项的 SwiGLU 模型在连续动态系统上的拟合稳定性。

3.3.1 构建数据集

阻尼震荡的物理方程为 $$y(t) = A e^{- t}cos(t +)

- 物理到数据的映射：
  - 利用滑动窗口，我们将连续的动力学系统演化，转化为了标准的监督学习任务（给出前因，预测后果）。这没有增加模型的参数量，而是极大地丰富了模型观察系统状态的“切片”数量。
- 工程解耦：
  - Dataset 只负责回答“数据有多长”和“第 i 个数据长什么样”
  - DataLoader 负责统筹全局，接管了打乱顺序、组装批次（Batching）以及未来可能用到的多进程数据加载。

[17]:
```
import torch
from torch.utils.data import Dataset, DataLoader
class DampedOscillatorDataset(Dataset):
    def __init__(self, dots,A,gamma,omega,phi):
        self.dots = dots
        self.A = A
        self.gamma = gamma
        self.omega = omega
        self.phi = phi
        self.t = torch.linspace(0,20,self.dots)
        self.y = torch.exp(-self.gamma * self.t) * self.A * torch.sin(self.
 ↪omega * self.t + self.phi)
        self.y_unfolded = self.y.unfold(0, 11, 1) # 生成长度为 11、步长为 1 的滑
动窗口 (因为我们要用前 10 个点预测第 11 个点，所以窗口长度是 11)
    def __len__(self):
        return self.dots - 10  # 因为最后 10 个点无法形成完整的输入输出对
    def __getitem__(self, idx):
        return self.y_unfolded[idx, :10], self.y_unfolded[idx, 10]  # 前 10 个点
作为输入，第 11 个点作为目标输出
```

[18]:
```
# 实例化
dataset = DampedOscillatorDataset(dots=1000,A=1.0,gamma=0.1,omega=2*3.
 ↪1415,phi=0)
train_loader = DataLoader(dataset, batch_size=32, shuffle=True) # 创建数据加载
器，每个批次 32 个样本，打乱顺序
```

3.3.2 封装训练引擎

[58]:
```
class Trainer:
    def __init__(self, model, dataloader, optimizer, task_loss_fn,␣
 ↪ma_penalty_module, epochs=100, device='cpu'):
        self.model = model.to(device)
        self.dataloader = dataloader
        self.optimizer = optimizer
        self.task_loss_fn = task_loss_fn
        self.ma_penalty_module = ma_penalty_module
        self.epochs = epochs
```

```
        self.device = device
        self.history = {"task_loss": [], "penalty_loss": []}

    def _train_step(self, batch_x, batch_y):
        # 处理单个批次的数据
        self.optimizer.zero_grad()
        y_pred = self.model(batch_x)
        batch_y = batch_y.view(-1, 1)  # 确保目标输出的形状与预测输出匹配
        task_loss = self.task_loss_fn(y_pred, batch_y)
        penalty_loss = self.ma_penalty_module()
        total_loss = task_loss + penalty_loss
        total_loss.backward()
        self.optimizer.step()
        self.history["task_loss"].append(task_loss.item())
        self.history["penalty_loss"].append(penalty_loss.item()/self.
↪ma_penalty_module.extra_loss_weight)  # 记录原始的惩罚损失（不乘权重）

    def train(self):
        print(f"开始在设备 {self.device} 上进行{self.epochs} 轮训练...")
        for epoch in range(self.epochs):
            self.model.train()
            for batch_x, batch_y in self.dataloader:
                batch_x, batch_y = batch_x.to(self.device), batch_y.to(self.
↪device)
                self._train_step(batch_x, batch_y)
        print("训练完成！ ")
```

3.3.3 主程序

[59]:
```
import torch
import torch.nn as nn
from SwiGLU import SwiGLU
from MAPenalty import MAPenaltyLoss
class DampedOscillatorExperiment:
    """ 阻尼谐振子实验封装类"""
```

```python
    def __init__(self, seed=42, device=None):
        self.seed = seed
        self.device = device or ('mps' if torch.backends.mps.is_available()␣
↪else 'cpu')

        # 实验组件（延迟初始化）
        self.dataset = None
        self.train_loader = None
        self.model = None
        self.ma_penalty = None
        self.optimizer = None
        self.trainer = None

    def setup(self,
              dots=1000, A=1.0, gamma=0.1, omega=6.283, phi=0,
              batch_size=64, d_model=128, d_hidden=512,
              k=5, power_iters=10, penalty_weight=1e-2,
              lr=1e-3):
        """ 初始化所有实验组件"""
        torch.manual_seed(self.seed)

        # 数据集
        self.dataset = DampedOscillatorDataset(dots=dots, A=A, gamma=gamma,␣
↪omega=omega, phi=phi)
        self.train_loader = DataLoader(self.dataset, batch_size=batch_size,␣
↪shuffle=True)

        # 模型
        self.model = nn.Sequential(
            nn.Linear(10, d_model),
            SwiGLU(d_model=d_model, d_hidden=d_hidden),
            SwiGLU(d_model=d_model, d_hidden=d_hidden),
            nn.Linear(d_model, 1)
        ).to(self.device)

        # MA 惩罚模块（绑定到第一个 SwiGLU 层）
```

```python
        self.ma_penalty = MAPenaltyLoss(
            target_module=self.model[1],
            k=k,
            power_iters=power_iters,
            extra_loss_weight=penalty_weight
        )

        # 优化器
        self.optimizer = torch.optim.AdamW(self.model.parameters(), lr=lr)

        return self

    def train(self, epochs=100):
        """ 执行训练"""
        if self.model is None:
            raise RuntimeError("请先调用 setup() 初始化实验组件")

        self.trainer = Trainer(
            model=self.model,
            dataloader=self.train_loader,
            optimizer=self.optimizer,
            task_loss_fn=nn.MSELoss(),
            ma_penalty_module=self.ma_penalty,
            epochs=epochs,
            device=self.device
        )
        self.trainer.train()

        # 清理 hook
        self.ma_penalty.handle.remove()
        return self

    def save_model(self, path='damped_oscillator_weights.pth'):
        """ 保存模型权重"""
        if self.model is None:
            raise RuntimeError("模型未初始化")
```

```
        torch.save(self.model.state_dict(), path)
        print(f"模型已保存至 {path}")
        return self

    def load_model(self, path='damped_oscillator_weights.pth'):
        """ 加载模型权重"""
        if self.model is None:
            raise RuntimeError("模型未初始化")
        self.model.load_state_dict(torch.load(path, map_location=self.device))
        print(f"模型已从 {path} 加载")
        return self

    def evaluate(self):
        if self.model is None:
            raise RuntimeError("模型未初始化")
        self.model.eval()
        with torch.no_grad():
            t = self.dataset.t
            y_true = self.dataset.y
            initial_input = y_true[:10].to(self.device).unsqueeze(0)  # 取前 10
个点作为初始输入
            current_input = initial_input.clone()  # 最近 10 个点作为输入
            predictions = initial_input.squeeze(0).cpu().numpy().tolist()  # 从
Tensor 转换为列表
            for _ in range(10, len(t)):
                y_next = self.model(current_input)
                predictions.append(y_next.item())  # 将预测结果添加到列表中
                current_input=torch.cat([current_input[:,1:], y_next.
↪view(1,-1)], dim=1)  # 更新输入，丢弃最旧的点，添加最新的预测
        # 绘图对比
        import matplotlib.pyplot as plt
        plt.figure(figsize=(12, 6))
        plt.plot(t.cpu().numpy(), y_true.cpu().numpy(), label="True Signal")
        plt.plot(t.cpu().numpy(), predictions, label="Predicted Signal",␣
↪linestyle='dashed')
        plt.xlabel("Time(t)")
```

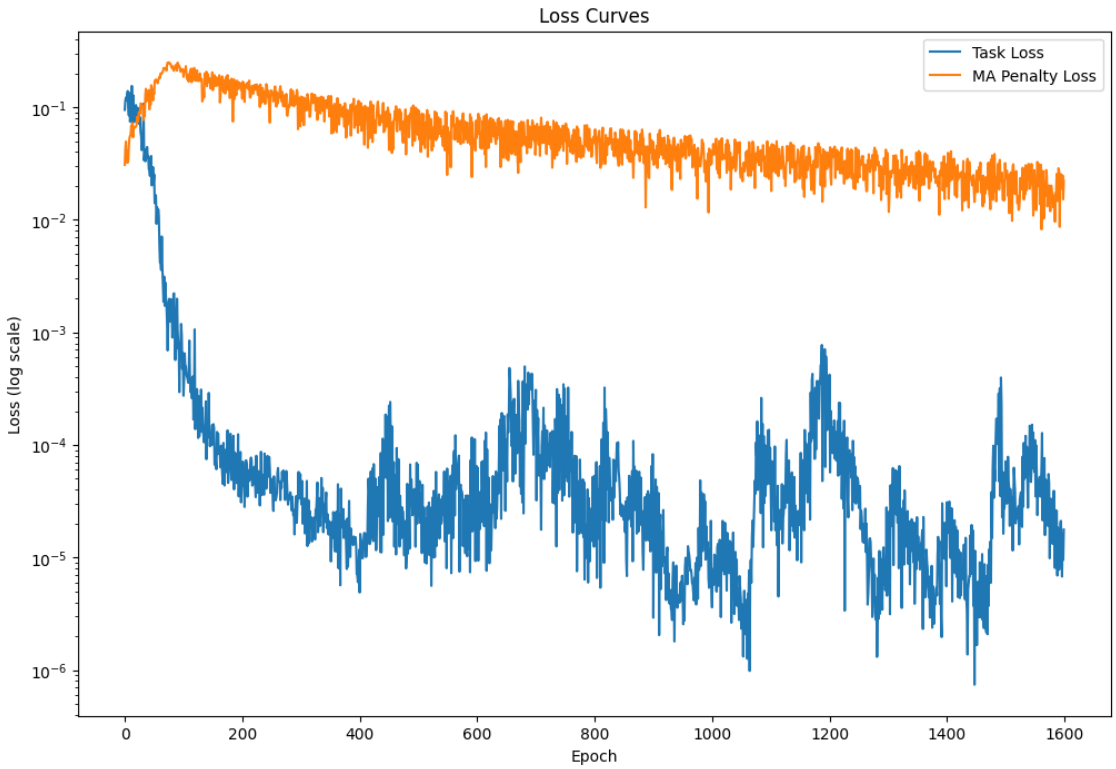
```
        plt.ylabel("Displacement(y)")
        plt.title("Damped Oscillator: True vs Predicted")
        plt.legend()
        plt.show()

    def plot_history(self):
        """ 绘制训练损失曲线"""
        if self.trainer is None:
            raise RuntimeError("尚未开始训练")

        import matplotlib.pyplot as plt
        plt.figure(figsize=(12, 8))
        plt.plot(self.trainer.history["task_loss"], label="Task Loss")
        plt.plot(self.trainer.history["penalty_loss"], label="MA Penalty Loss")
        plt.yscale("log")
        plt.xlabel("Epoch")
        plt.ylabel("Loss (log scale)")
        plt.title("Loss Curves")
        plt.legend()
        plt.show()
```

[60]:
```
experiment = DampedOscillatorExperiment(seed=42)
experiment.setup(
    dots=1000, A=1.0, gamma=0.1, omega=2*3.1415, phi=0,
    batch_size=64, d_model=128, d_hidden=512,
    k=5, power_iters=10, penalty_weight=1e-2, lr=1e-3
    )
```

[60]:
```
<__main__.DampedOscillatorExperiment at 0x118d19a90>
```

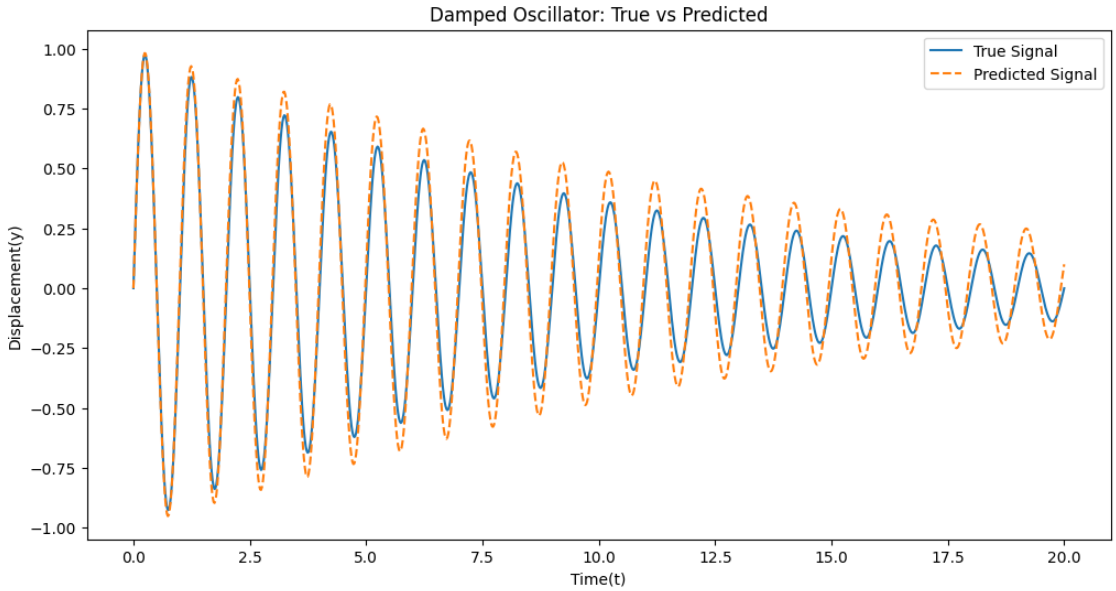
[62]:
```
experiment.train(epochs=100)
experiment.save_model()
experiment.plot_history()
```

```
开始在设备 mps 上进行 100 轮训练…
训练完成!
模型已保存至 damped_oscillator_weights.pth
```



[63]:
```
experiment.load_model()
experiment.evaluate()
```

模型已从 damped_oscillator_weights.pth 加载


Damped Oscillator: True vs Predicted
True Signal
Predicted Signal
Displacement(y)
Time(t)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0